ASSIGNMENT 1

1. Explain the difference between frontend, backend, and full-stack development with suitable real-world examples.

Type	Description	Technologies	Real-World Example
Frontend Development	Focuses on the visual part of a website — what users see and interact with.	HTML, CSS, JavaScript, React, Angular	When you open Facebook, the like button, profile layout, and news feed design are created by frontend developers.
Backend Development	Handles data processing, logic, and communication with databases and servers.	Node.js, Python (Django/Flask), Java (Spring), PHP, SQL	When you click "Post" on Facebook, the backend saves your post in a database and retrieves it when someone views your profile.
Full-Stack Development	Combines both frontend and backend development skills to build complete web applications.	MERN (MongoDB, Express, React, Node.js), MEAN (MongoDB, Express, Angular, Node.js)	A full-stack developer can build both the Facebook interface and the backend system that stores posts and comments.

2. Create a simple diagram showing how the client-server model works in web architecture.

```
[User / Client Browser]
        |
        |  (HTTP Request)
        v
[Web Server / Backend]
        |
        |  (Database Query)
        v
```

```
     [Database]
         |
         |  (Response Data)
         v
  [Web Server sends response]
         |
         v
  [Client displays web page]
```

3. Describe how a browser requests and displays a web page from a web server.

When you type a website URL in a browser, the browser sends a request to the web server using HTTP or HTTPS. The server then finds the requested page and sends it back to the browser. The browser receives the HTML file, then downloads related resources like CSS, JavaScript, and images. It renders the page step by step: first structure (HTML), then styling (CSS), and finally interactivity (JavaScript). This process is known as rendering or page loading.

4. Identify and list the tools required to set up a web development environment. Explain the purpose of each.

Tool	**Purpose**
VS Code	Code editor used to write HTML, CSS, and JavaScript.
Web Browser (Chrome, Firefox)	Used to test and view web pages.
Node.js	Enables running JavaScript on the server and installs packages using npm.
Git & GitHub	Version control and collaboration tools for tracking project changes.
Live Server Extension (VS Code)	Automatically refreshes the browser when you save files.

5. Explain what a web server is and give examples of commonly used servers.

A web server is software or hardware that delivers web content to users over the internet. It stores website files (HTML, CSS, images, etc.) and serves them when requested by clients. Popular web servers include:
- Apache HTTP Server – Open-source and widely used.
- Nginx– Known for speed and handling high traffic.
- Microsoft IIS – Integrated with Windows systems.
- LiteSpeed – Efficient and lightweight alternative.

6. Define the roles of a frontend developer, backend developer, and database administrator in a project.

- Frontend Developer: Designs and implements the visual layout and user interactions. They ensure the website looks appealing and works on all devices.
- Backend Developer:Manages the server-side logic, APIs, and database interactions. They make sure data flows correctly between the client and server.
- Database Administrator (DBA): Handles the design, security, and maintenance of databases. They ensure data integrity, backups, and performance optimization.

7. Install VS Code and configure it for HTML, CSS, and JavaScript development. Take a screenshot of the setup.

To set up VS Code for HTML, CSS, and JavaScript development:
1. Download and install Visual Studio Code from https://code.visualstudio.com.
2. Install extensions like ‘Live Server’, ‘Prettier’, and ‘JavaScript (ES6) snippets’.
3. Create a project folder and open it in VS Code.
4. Create HTML, CSS, and JS files for your project.

After setting up, take a screenshot of your workspace showing all three files open and Live Server running.

8. Explain the difference between static and dynamic websites. Provide an example of each.

A static website displays fixed content that doesn’t change unless the developer manually updates it. These sites are fast and simple, suitable for portfolios or company profiles. Example: A personal portfolio website.

A dynamic website generates content based on user interaction or data from a database. They use server-side scripting languages like PHP or Node.js. Example: Facebook or YouTube, where content changes for each user.

9. Research and list five web browsers. Explain how rendering engines differ between them.

Five popular web browsers are:
1. **Google Chrome** – Uses Blink rendering engine.
2. **Mozilla Firefox** – Uses Gecko engine.
3. **Microsoft Edge** – Uses Blink (formerly EdgeHTML).
4. **Apple Safari** – Uses WebKit engine.
5. **Opera** – Also uses Blink engine.

Rendering engines are responsible for converting HTML, CSS, and JavaScript into visual content. Each engine may interpret and optimize elements differently, which is why websites may look slightly different across browsers.

10. Draw a labeled diagram showing the basic web architecture flow — client, server, database, and APIs.

```
+-------------------+
|     CLIENT        |
| (Browser, App)    |
+-------------------+
        |
        |  HTTP Request
        v
+-------------------+
|    WEB SERVER     |
| (Node.js, Apache) |
+-------------------+
        |
        |  API Call / Query
        v
+-------------------+
|     DATABASE      |
```

```
| (MySQL, MongoDB)  |
+-------------------+
         |
         |  Response
         v
+-------------------+
|   CLIENT DISPLAY  |
| (Rendered Page)   |
+-------------------+
```